BIRD SONG GENERATION USING DEEP LEARNING

DEPARTMENT OF INFORMATION TECHNOLOGY

Submitted by:

Mayank Singh (2021UIT3030)

Sakib Hassan (2021UIT3039)

Abhay Prajapati (2021UIT3058)

Under the supervision of

Dr. Mohit Sajwan

DEPARTMENT OF INFORMATION TECHNOLOGY

NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY

Dec, 2024

DECLARATION

Department of Information technology

Delhi-110078, India

We, Mayank Singh (2021UIT3030), Sakib Hassan (2021UIT3039), Abhay Prajapati (2021UIT3058) of B. Tech., Department of Information Technology, hereby declare that the Project I -report titled “BIRD SOUND GENERATION USING DEEP LEARNING” which is submitted by us to the Department of Information Technology, Netaji Subhas University of Technology, is original and not copied from source without proper citation. This work has not previously formed the basis for the award of any Degree.

Place: Delhi
Date: 8/12/24

Mayank Singh

Sakib Hassan

Abhay Prajapati

CERTIFICATE

Department of Information Technology

Delhi-110078, India

This is to certify that the work embodied in the Project I-Report titled "BIOPHONIC SPECIES SOUND GENERATION USING DEEP LEARNING" has been completed by Mayank Singh (2021UIT3030), Sakib Hassan (2021UIT3039), Abhay Prajapati (2021UIT3058) of B.Tech., Department of Information Technology, under my guidance. This work has not been submitted for any other diploma or degree of any University.

Place: Delhi

Date: 8/12/24

Dr Mohit Sajwan

Supervisor

INDEX

LIST OF FIGURES

List of Tables

ABSTRACT

Recent progress in neural audio generation has led to models that produce remarkably natural results. HiFi-GAN [1] leverages adversarial training to achieve high-fidelity waveform synthesis efficiently, while DDSP [2] incorporates differentiable signal processing elements to infuse musical and acoustic priors directly into the synthesis pipeline. Meanwhile, diffusion-based approaches, such as DiffWave [3] and WaveGrad [4], have demonstrated the potential for stable, high-quality audio generation through iterative denoising. Building on these foundations, FastDiff [5] emerges as a powerful conditional diffusion model that excels by integrating adaptive architectural components with strong conditioning signals.

Unlike GAN-based methods that rely on complex adversarial objectives, or DDSP's specialized signal processing layers, FastDiff applies a probabilistic denoising process guided by Mel-spectrogram conditioning and time embeddings. Its distinctive Time-Aware Location-Variable Convolution (LVC) layers dynamically adjust convolutional filters at each diffusion step, ensuring that noise removal and feature extraction adapt fluidly to both the conditioning features and the current noise level. This combination allows FastDiff to more precisely align generated audio with the given Mel-spectrogram's spectral-temporal cues, leading to improved fidelity and detail. Moreover, an optional noise predictor can reduce inference time by identifying a shorter noise schedule without compromising quality, making FastDiff computationally efficient relative to other diffusion-based models [6].

Beyond traditional text-to-speech tasks, FastDiff's ability to produce condition-specific, high-fidelity audio extends to domains like ecological sound analysis.

For instance, generating realistic bird call augmentations can enhance the robustness of classification models, improving biodiversity monitoring and environmental sound research pipelines. In this way, FastDiff not only outperforms or complements established approaches like HiFi-GAN, DDSP, and earlier diffusion models, but also broadens the practical applications of high-quality, condition-driven audio synthesis.

Chapter-1

Literary Survey

Reference	Methodology	Key Findings	Relevance to ECOGEN
Engel, J., Hantrakul, L., Gu, C., & Roberts, A. (2020)[2]	Differentiable Digital Signal Processing (DDSP) to synthesize audio through neural networks.	Introduced DDSP as a powerful tool for real-time audio synthesis with fine control over sound attributes.	Directly applicable to ECOGEN for generating bird songs with higher realism and flexibility.
Dhariwal, P., Nichol, A., Ramesh, A., Chen, P., Guo, M., et al. (2020)[11]	Jukebox: A VQ-VAE-based generative model for music conditioned on artist, genre, and lyrics.	Achieved high-fidelity music synthesis through a hierarchical VQ-VAE approach.	Can inspire hierarchical latent variable modeling in ECOGEN to improve bird song generation.
Payne, C. (2019)	MuseNet: A transformer-based model for generating music with multiple instruments and styles.	Demonstrated multi-track music generation with long-range dependencies.	MuseNet's approach could help ECOGEN with long-duration and multi-layered bird song generation.
Razavi, A., van den Oord, A., & Vinyals, O. (2019)[17]	VQ-VAE-2: A hierarchical generative model for high-fidelity image synthesis.	generated diverse, high-quality images using hierarchical of structureVQ-VAE-2.	Forms the foundation of ECOGEN's bird song generation using hierarchical latent space modeling.

Table 1.1

INTRODUCTION

OVERVIEW OF EXISTING ECOGEN ARCHITECTURE

1.1 MOTIVATION

The motivation for this work arises from the growing need to address several pressing issues in ecological research, particularly those concerning species conservation, biodiversity monitoring, and machine learning classification of bird vocalizations. Although advancements in acoustic monitoring technologies have significantly improved our ability to gather large amounts of data, significant challenges remain in the quality, balance, and representation of the datasets collected.

Data Imbalance in Ecological Datasets: One of the central issues in the domain of ecological acoustics is the imbalance in datasets across bird species. Many species are either under-represented or completely absent in the available recordings due to factors such as habitat degradation, geographic remoteness, or rarity of the species [10]. This scarcity of data for certain bird species limits the performance of machine learning models, which require large and diverse datasets for effective classification. For endangered or at-risk species, this lack of data further complicates conservation efforts, as accurate species identification becomes critical for habitat preservation and species protection.

Limitations of Traditional Augmentation Techniques: Existing techniques for overcoming data scarcity, such as data augmentation, have proven insufficient in generating the diversity required for robust model training. Methods like pitch shifting, time stretching, or adding

synthetic noise merely modify existing data without creating truly novel samples. This means that the machine learning models are still learning from a constrained set of audio features, limiting their ability to generalize across real-world conditions. Furthermore, many of these techniques fail to mimic the natural variability found in bird vocalizations, reducing their effectiveness in improving classification accuracy.

Benefits for Conservation and Research: From a conservation perspective, the ability to generate high-quality synthetic bird songs can have significant ecological impacts. For endangered or rare species, where capturing natural recordings may be impractical or impossible, synthetic audio can be used to enhance monitoring and detection systems. Additionally, having access to a richer dataset aids in behavior analysis and species-specific research, allowing ecologists to better understand vocalization patterns, breeding behaviors, and habitat preferences.

Contributing to the Field of Eco-acoustics: The development of ECOGEN as a tool to synthesize realistic bird songs, where monitoring biodiversity through sound is becoming increasingly important. This work not only addresses technical challenges in machine learning and data augmentation but also advances ecological conservation by providing a scalable method to generate lifelike bird songs that can aid in long-term biodiversity assessments.

1.2 Introduction to ECOGEN

ECOGEN, a framework that enables biologists to generate synthetic recordings of bird songs. To train the underlying **VQ VAE-2(Vector Quantized Variational AutoEncoder)** [14], an extensive data set containing hundreds of thousands of bird song patterns was used. It uses a codebook with 512 keys to store abstract features (e.g. style, pitch, tone, background noise and rain) that are commonly present across all species. After training, It was observed that ECOGEN's codebook uses more than 95.00% of the available keys, indicating its ability to capture a wide range of learned bird song patterns [10].

1.2.1 Working of ECOGEN

In the current ECOGEN model, the architecture involves **two main stages**:

Stage 1: Preprocessing: A VQ-VAE2 (Vector Quantized Variational Autoencoder) model converts Bird songs into spectrograms. The model learns latent representations of the bird calls in the form of discrete codes.

Stage 2: Spectrogram Generation and Reconstruction: New spectrograms are generated using the latent codes, which are then reconverted back into audio using methods like the Griffin-Lim algorithm.

1.3 Limitations of Existing Architecture

1. Slow Processing: Converting between time-domain waveforms and frequency-domain spectrograms introduces additional computational step.
2. Limited Control: Spectrogram-based methods don’t allow for direct control over audio features like pitch, loudness, and timbre.
3. Loss of Audio Quality: The conversion can lose some of the quality and details of the sound.

1.4 Dataset

1.4.1 Data Source

The training data set was built from a selection of bird song recordings coming from the Xeno-Canto database (Kaggle)[4][5].

The data set contains 23,784 bird recordings from around the world across 264 species.These recordings were recorded in the birds' natural environment, with distinct environmental background noises (e.g. wind, flowing water, rain etc.) present in the recordings [4][5].

Aldfly.mp3 [sample from Xeno-Canto database (Kaggle)][12][13]]

Example: Alder Flycatcher

1.4.2 Preprocessing

Data set was pre-processed by segmenting the bird recordings into 2-s segments with a 1-s overlap. This segmentation resulted in a total of 2,099,153 unique recording samples, each of which served as input for the model during the training phase[10].

1.5 Core Components of ECOGEN

1.5.1 VQ-VAE2 (Vector Quantized Variational AutoEncoder)

The VQ-VAE2 is an auto-encoder architecture comprising an encoder and a decoder network. The encoder first maps an input x to quantized latent variables z, also known as codebook; the decoder then reconstructs the input x from the codebook .The codebook contains essential information that enables the decoder to accurately reconstruct the input [21][22]. Quantization is performed on the codebook using a nearest neighbors' clustering algorithm, which groups similar patterns together[21].

Essentially, the codebook in the VQ-VAE2 architecture represents the presence of specific features in the input data through assigned key IDs. For instance, a particular bird call pattern in the audio file might be assigned key number 10. Whenever this pattern is detected in any audio file, the model will assign it to key number 10 [21][22].

The codebook space is structured so that similar patterns are assigned to the same or nearby clusters, while dissimilar features are placed farther apart. In other words, the VQ-VAE2 network learns to decompose bird song patterns, capturing the most relevant ones and organizing them in a features catalog (the codebook) [21].

1.6 Spectrogram Representations

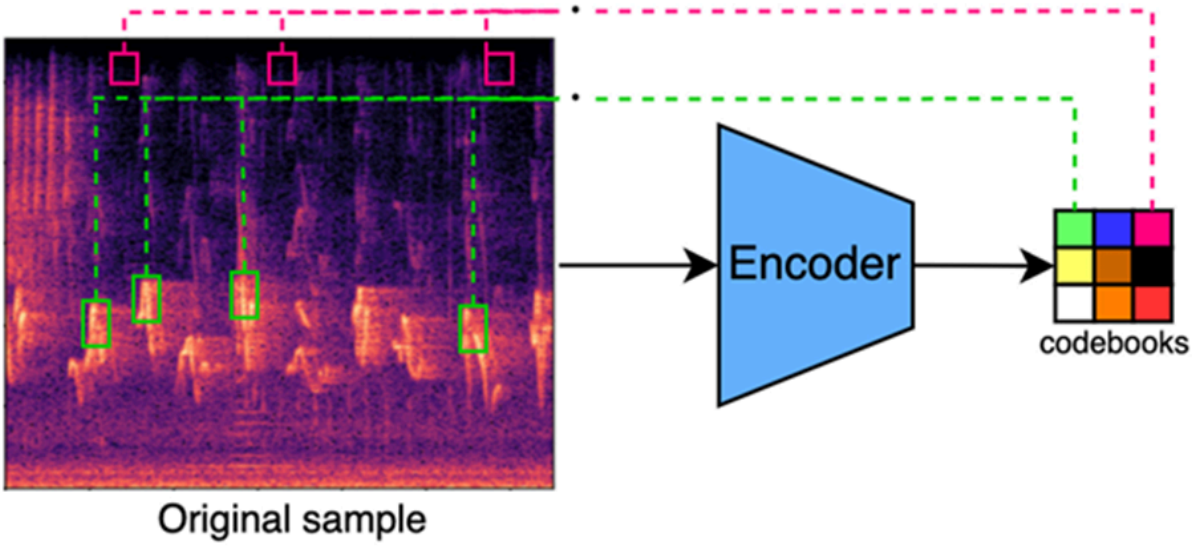

Figure 1.1 audio encoding to codebook (from ECOGEN... [10])

1.6.1. Spectrogram:

A spectrogram is generated by applying the short-time Fourier transform (STFT) to the audio signal.It is reversible, allowing us to convert the audio back to its waveform from the compact representation, and vice versa [23][24].

This process produces a two-dimensional grid, akin to an image, that depicts the signal's frequency variations over time. The spectrogram can be transformed back into a digital audio signal by employing the Griffin and Lim method for phase estimation, followed by the inverse short-time Fourier transform[23][24]

1.6.2 Mel-spectrogram:

A Mel-spectrogram builds on the standard spectrogram by applying a nonlinear transformation to the frequency axis, mapping it to the Mel scale, which is more aligned with human auditory perception. This representation highlights key temporal and spectral features in the signal, making it particularly useful for machine learning models like CNNs in tasks such as audio analysis and generation. In models like ECOGEN, Mel-spectrograms are used as input to learn and generate bird songs, which can then be converted back to waveforms using methods such as Griffin-Lim or advanced approaches like iterative phase reconstruction algorithms for higher fidelity [10][23]

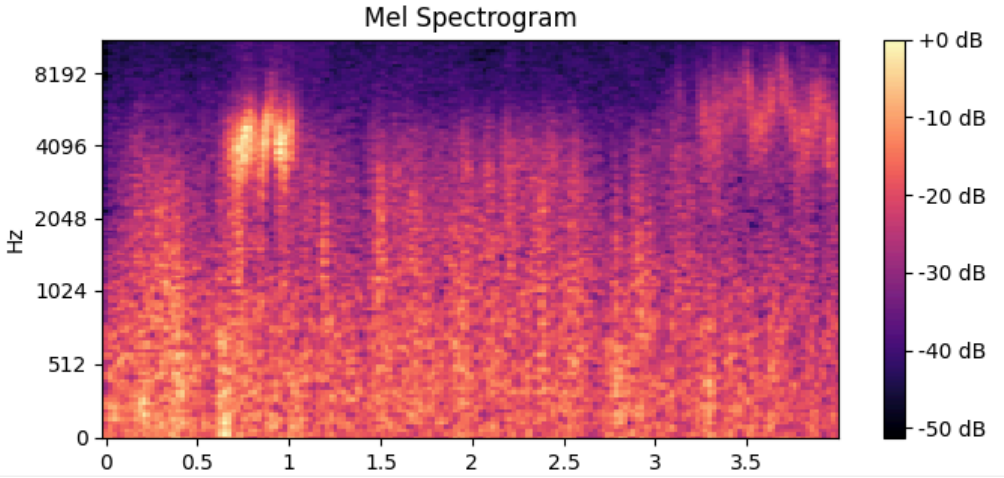

Figure 1.2 Mel-Spectrogram of Alferfly (from ECOGEN... [10])

1.7 Augmentation Techniques

To enhance training and improve robustness, ECOGEN employs the following augmentation strategies:

- **Interpolation:** Smoothly transitions between features for generating novel patterns.
- **Noise Perturbation:** Introduces environmental noise variations to make the model robust to diverse input conditions.

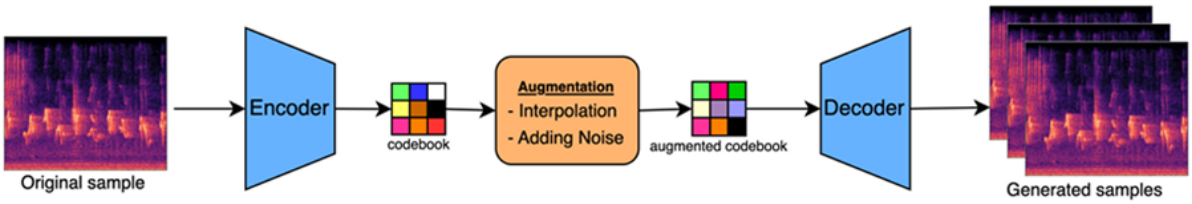

Figure 1.3 augmentation process(from ECOGEN... [10])

CHAPTER-2

PROPOSED METHODOLOGY-1

Differentiable Digital Signal Processing (DDSP)

2.1 Introduction to Differentiable Digital Signal Processing (DDSP)

Differentiable Digital Signal Processing (DDSP) represents an innovative approach to audio synthesis that integrates the structured interpretability of classical digital signal processing (DSP) elements with the adaptive capabilities of deep learning models [7]. This methodology addresses a critical challenge in neural audio generation by bridging the gap between opaque neural network architectures and more transparent DSP techniques, ultimately enabling high-quality and naturalistic sound synthesis.

The core innovation of DDSP lies in its ability to combine traditional DSP components like filters, oscillators, and reverberation modules with neural network learning paradigms [7][26]. By making these signal processing elements differentiable, researchers can now train neural networks to understand and generate complex audio signals with unprecedented interpretability and precision.

2.2 Overview of DDSP Framework

2.2.1 Motivation for DDSP

Traditional methods for audio synthesis, such as WaveNet and spectrogram-based models, face challenges:

- **WaveNet**: High computational cost, generating audio one sample at a time,slow at generating sound.
- **Spectrogram Methods**: Loss of phase information and artifacts during reconstruction.

In contrast, DDSP synthesizes audio directly in the time domain, preserving phase information and offering precise control over pitch, loudness, and timbre.

2.2.2 What can we do with it?

Using differentiable oscillators, filters, and reverberation from the DDSP library enables us to train high-quality audio synthesis models with less data and fewer parameters, as they do not need to learn to generate audio from scratch. For example in the samples below, all the models were trained on less than 13 minutes of audio. Also, since the DDSP components are interpretable and modular, we can extend to a range of fun behaviors such as blind dereverberation, extrapolation to pitches not in the training set, and timbre transfer between disparate sources. Below are some examples of timbre transfer, keeping the frequency and loudness the same and changing the character of the instrument's sound, including tuning a singer's voice note to a violin[7][26].

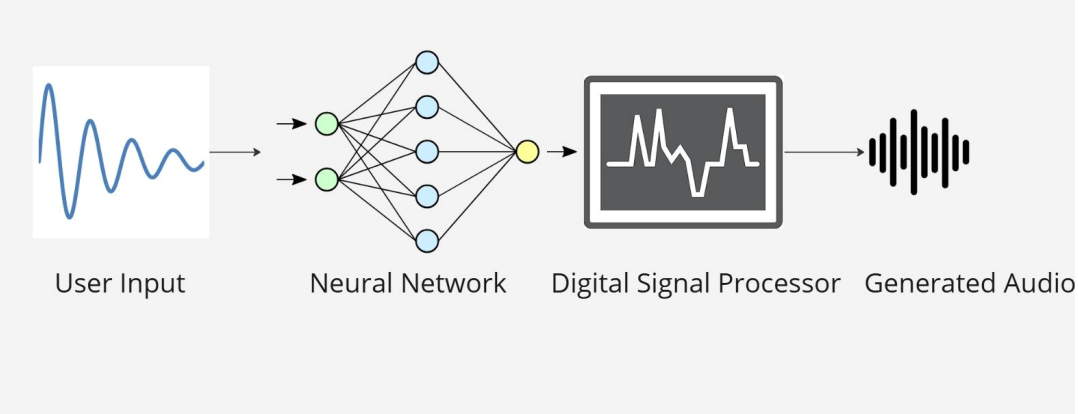

Figure 2.1

2.2.2 Key Concepts in DDSP

1. **Harmonic Additive Synthesizer**: Generates audio by summing sinusoids at multiple frequencies.

$$x(t)=k=1\sum KAk\sin(2\pi fkt+\phi k) \qquad (2.1)$$

A_k, f_k, and ϕ_k are each sinusoid's amplitude, frequency, and phase.

2. **Subtractive Noise Synthesizer**: employs time-varying filters to filter out white-noise:

y(t)=Filter(xnoise,h(t))

where h(t) represents filter parameters controlled by a neural network.

3. **Reverberation Module**: Adds spatial characteristics to synthesized audio.
4. **Differentiability**: All components are differentiable, enabling optimization using backpropagation.

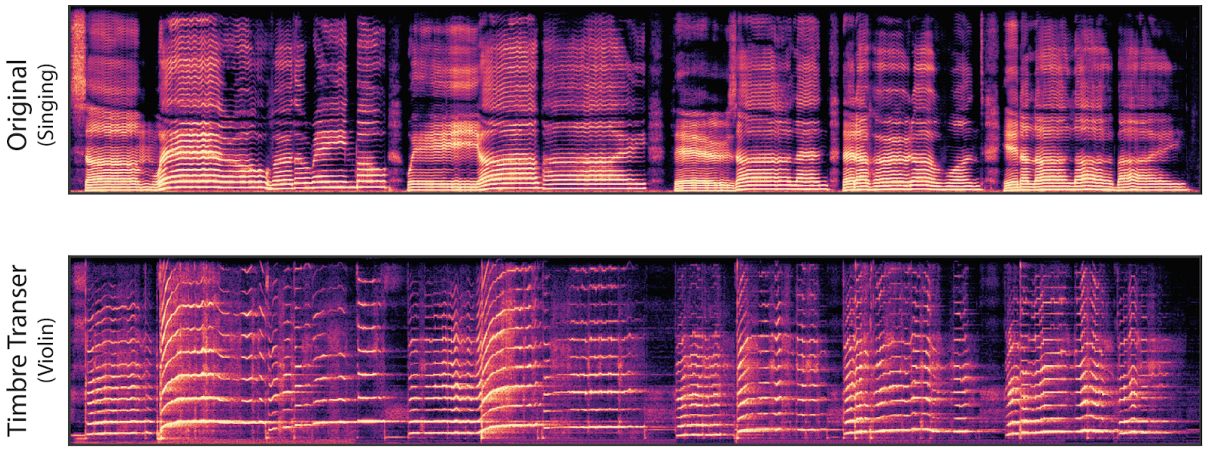

Figure 2.2 Image Shows Timer Transfer of a singing audio to violin audio

2.3 DDSP for Bird Sound Generation

2.3.1 Challenges in Bird Sound Generation

- Limited data availability for endangered species.
- High computational cost of traditional models (e.g., WaveNet).
- Artifacts introduced by spectrogram inversion methods.
- Preprocessing is expensive

2.3.2 Advantages of DDSP for Bird Sounds

- **Phase Preservation**: Ensures high fidelity by avoiding spectrogram inversion.
- **Fine Control**: Enables precise manipulation of features like pitch, loudness, and harmonic content.
- **Faster Sound Generation:** DDSP in comparison to other generation models like wavenet or VQ-VAE2 is much faster in new sound generation

2.4 Implementation of DDSP for Bird Sound Generation

2.4.1 Input Processing and Feature Extraction

1. Conversion of audio data into .wav form so that training captures all details and mapping of audio to species
2. Audio recordings are analyzed to extract features such as pitch and loudness.
    a. **Pitch Estimation**:

$$f0(t)=ArgMax(FFT(x(t))) \quad (2.2)$$

    b. **Loudness Computation:**

$$L(t)=10log10(N1i=1\sum Nxi2(t) \quad (2.3)$$

3. These features serve as inputs to the DDSP model for audio synthesis.

2.4.2 DDSP Module Architecture

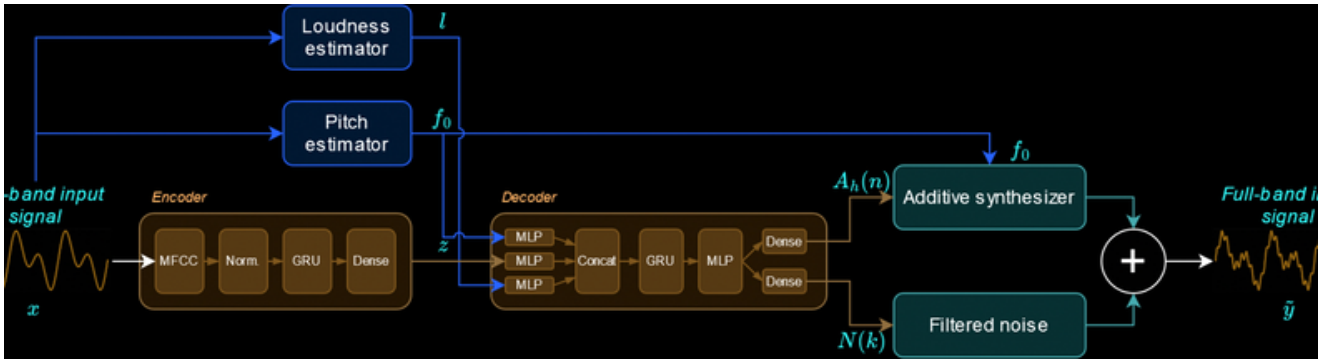

Figure 2.3 DDSP architecture for bird sound generation

- **Autoencoder Framework**: incorporates a noise subtractive synthesiser and a additive harmonic synthesiser into the decoder.
- **Loss Function**: Spectrogram comparison across multiple frame sizes:

$$L = \sum_{i=1}^{N} \| S(x_{gen}) - S(x_{true}) \|_2^2 \qquad (2.4)$$

Where S(x) represents the spectrogram.

2.4.3 Computational Efficiency

DDSP significantly reduces computation time, synthesizing high-fidelity audio faster than traditional methods.

2.5 Results and Applications

2.5.1 Bird Sound Generation Results

- **Quality**: DDSP-generated bird songs demonstrate natural timbre and clarity.
- **Efficiency**: Generates minutes of audio in seconds.

2.5.2 Applications in Ecology

- Enabling acoustic monitoring for rare or endangered bird species.
- Augmenting datasets for bird classification models.

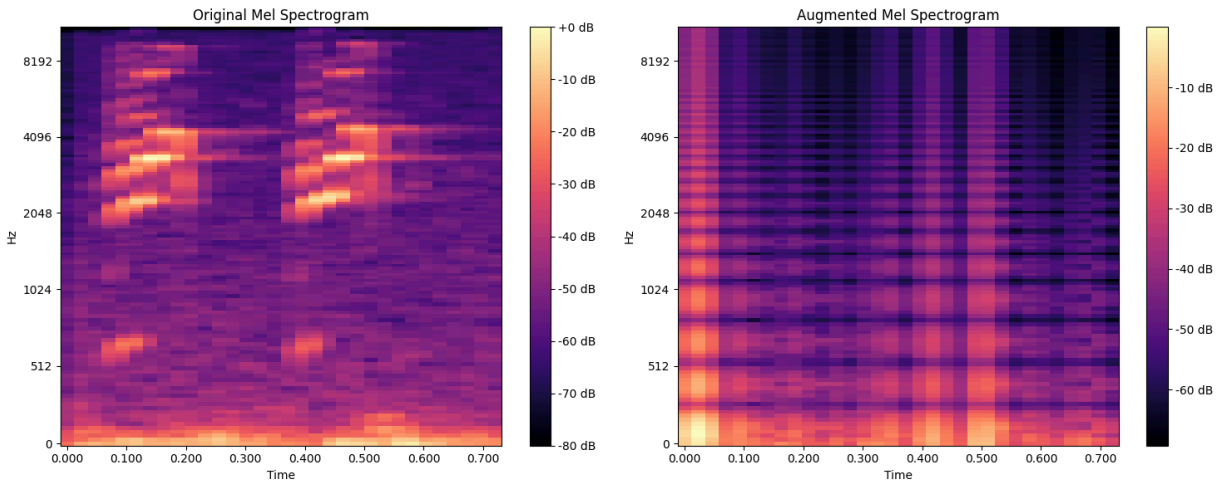

Figure 2.4 Mel Spectrogram Comparison of Original audio vs generated audio of species ‘canwar’

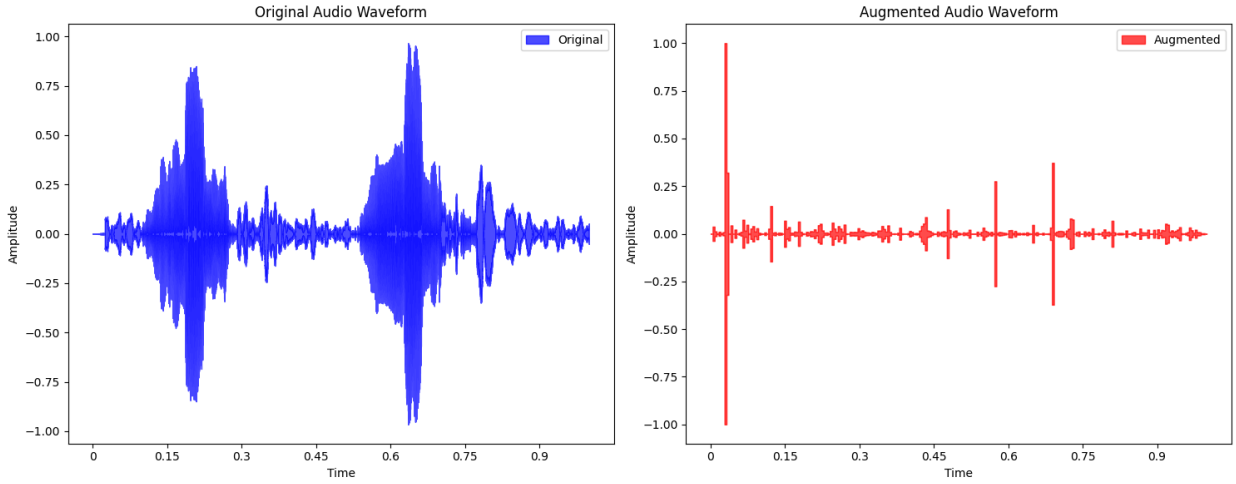

Figure 2.5 waveform of original audio(left) vs generated audio(right)

CHAPTER-3

PROPOSED METHODOLOGY-II

FAST-DIFFUSION

3.1 Background

Data augmentation is vital for machine learning tasks, particularly in bioacoustics, where obtaining labeled datasets is challenging. **FastDiff**, a conditional diffusion probabilistic model, offers a robust method for generating synthetic bird audio to augment datasets, improving downstream tasks like species identification and ecological monitoring [5].

3.2 Objective

We explore adapting FastDiff for bird sound generation by modeling the distribution of bird calls and leveraging its conditional synthesis capabilities. FastDiff combines diffusion models with adaptive convolutional techniques and efficient inference strategies to generate high-fidelity bird audio. By augmenting datasets with realistic synthetic samples, it enhances the robustness and performance of bird sound analysis models.

3.2 Methodology

3.2.1 Overview of Diffusion Probabilistic Models

Diffusion probabilistic models (DPMs) generate samples by approximately inverting a progressive noise addition procedure [15]. These models comprise three primary components: a forward (noising) process, a reverse (denoising) process, and a final sampling stage.

3.2.2 Forward Diffusion Process

The forward diffusion process takes an initial clean data sample x0x_0x0 drawn from the data distribution q(x0) and iteratively adds Gaussian noise over a sequence of steps

$t \in \{1,\ldots,T\}$ Each forward step applies the following transition:

$$q(xt \mid xt-1)=N(xt;1-\beta t\ xt-1,\beta tI),$$

where βt\beta_tβt denotes the noise variance at step ttt. After sufficiently many steps, the distribution of xTx_TxT approaches a simple Gaussian N(0,I)[5] **(3.1)**

where:

- $t \in \{1,\ldots,T\}$ is the timestep,
- βt is the noise variance at step ttt.

After T steps, the sample xT becomes approximately Gaussian:

$$\mathbf{q(xT)\approx N(0,I).}$$

3.2.3 Reverse Sampling Process

The reverse process reconstructs the original data x0 from Gaussian noise xT by iterative denoising:

$$p\theta(xt-1 \mid xt)=N(xt-1;\mu\theta(xt,t),\Sigma\theta(xt,t))$$

where μθ and Σθ are neural network outputs parameterized by θ.[5] **(3.2)**

3.3 Training Objective

The model predicts the noise ϵ added during forward diffusion. The loss function minimizes the mean squared error (MSE) between true noise ϵ and predicted noise ϵθ:

$$xt=\alpha t^{-}x0+1-\alpha t^{-}\epsilon, \quad \textbf{(3.3)}$$

$$L(\theta)=Ex0,t,\epsilon[\ \| \ \epsilon-\epsilon\theta(xt,t)\ \| \ 2].$$

This objective ensures that the model learns to remove the noise step-by-step, eventually recovering the original clean data [5]. **(3.4)**

3.4 FastDiff: Enhancements for Audio Generation

3.4.1 Conditioning on Mel-Spectrograms

3.4.1 Conditioning Using Mel-Spectrograms

FastDiff incorporates mel-spectrograms as side information to guide audio generation. The conditional model thus becomes:

$$p\theta(x0 \mid c)$$

where ccc is the conditioning mel-spectrogram extracted from real bird recordings. This ensures that the generated waveform conforms to the frequency and temporal characteristics captured by the spectrogram.[5]

.

3.4.2 Time Embedding

To capture timestep information during training and inference, sinusoidal positional encodings map t to a high-dimensional embedding vector TE(t):

$$TE(t)=(\sin(\omega 1t),\cos(\omega 1t),\sin(\omega 2t),\cos(\omega 2t),\ldots).$$

providing a rich representation of step indices [5]. **(3.6)**

3.4.3 Time-Aware Location-Variable Convolution (LVC)

LVC dynamically adapts convolutional filters based on time embeddings TE(t) and spectrogram conditions ccc. For input x:

$$x' = \text{PointwiseConv}(\text{DepthwiseConv}(x) \odot K\theta(c, TE(t))), \quad (3.7)$$

where:

where Kθ(c,TE(t))K_\theta(c, TE(t))Kθ(c,TE(t)) generates kernels conditioned on ccc and TE(t)TE(t)TE(t), and ⊙\odot⊙ denotes element-wise multiplication. This approach ensures that the model's convolutional operations vary adaptively with time and acoustic context [5].

3.4.4 Noise Predictor for Fast Inference

A key innovation in FastDiff is the introduction of a noise predictor to shortcut the full diffusion process. By estimating a suitable number of timesteps T′<TT' < TT′<T, the model can initiate the reverse process from a less noisy intermediate state xT', thereby cutting down on the number of reverse denoising steps needed. This significantly speeds up inference time without compromising audio fidelity.

3.5 Implementation Pipeline

3.5.1 Data Preparation

1. **Audio Collection**: Real bird recordings were sourced from bioacoustic repositories.
2. **Feature Extraction**: Mel-spectrograms were extracted to serve as conditioning inputs.

3.5.2 Model Architecture

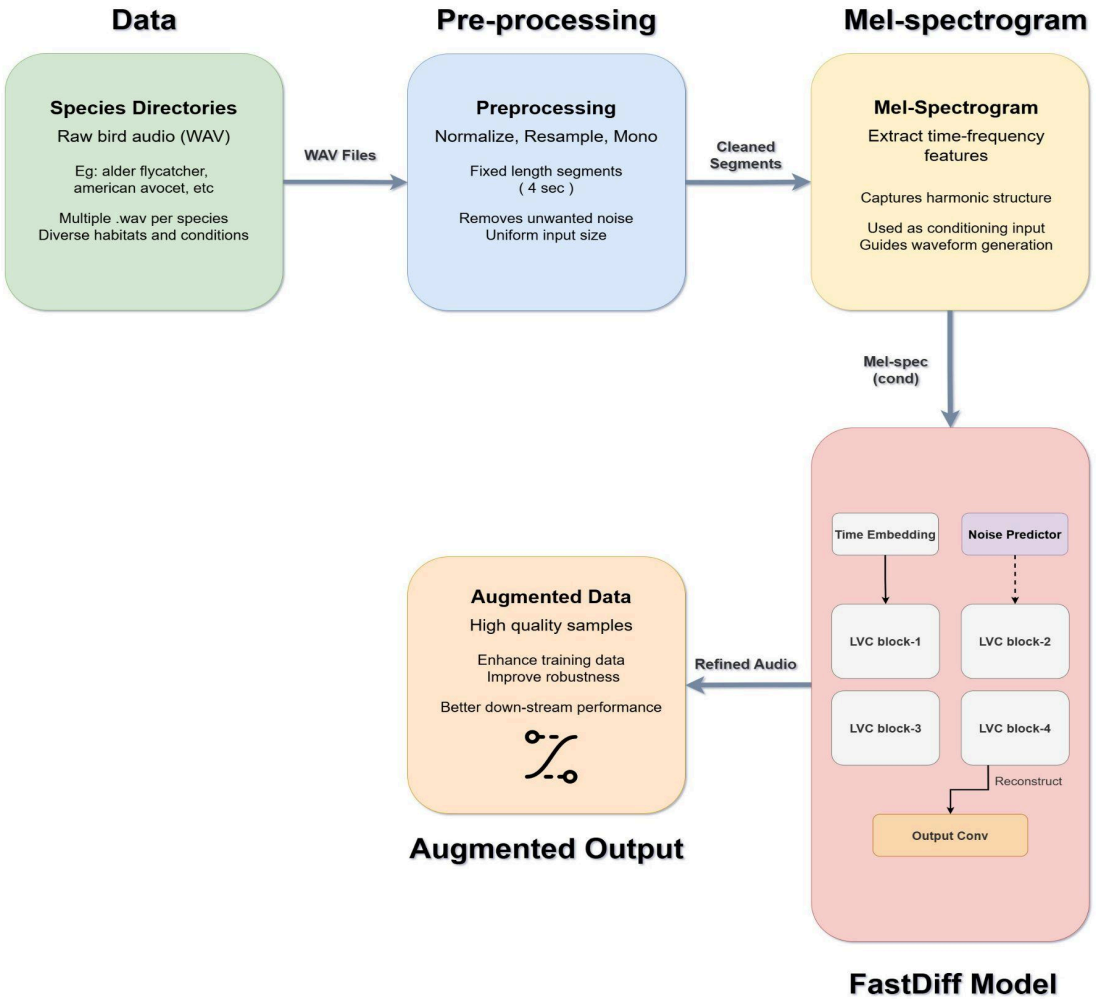

Figure 3.1 Architecture of FastDiff

3.6 Training Process

1. Input Preparation:
    - Pair (x0,c) for each training sample.
    - Generate noisy samples xt using the forward process:

$x_t = \bar{\alpha}_t x_0 + 1 - \bar{\alpha}_t \epsilon$, [5] **(3.8)**

where $\bar{\alpha}_t = \prod_{i=1}^{t}(1-\beta_i)$

2. Noise Prediction:
3. Train the model to predict ϵ using the loss:

$L(\theta) = E_{x0,t,\epsilon}[\| \epsilon - \epsilon_\theta(x_t,t,c) \|^2]$ [5]

3.7 Inference Process

1. Noise Predictor:
    - Estimate the noise schedule.
    - Directly predict the noise schedule
2. Reverse Diffusion:
    - Start with $x_T \sim N(0,I)$.
    - Iteratively compute:

$x_{t-1} = \alpha_t^{-1}(x_t - 1 - \bar{\alpha}_t \beta_t \epsilon_\theta(x_t,t,c)) +$ additional noise [6]

3.8 Results

3.8.1 Introduction

1. **Generated Audio Quality**:
    - Examples of synthetic bird calls matching real Mel-spectrogram patterns.
2. **Model Performance**:
    - Metrics: spectrogram similarity, classification accuracy on augmented datasets.
3. **Inference Speed**:
    - Comparison of standard diffusion vs. FastDiff with noise predictor.

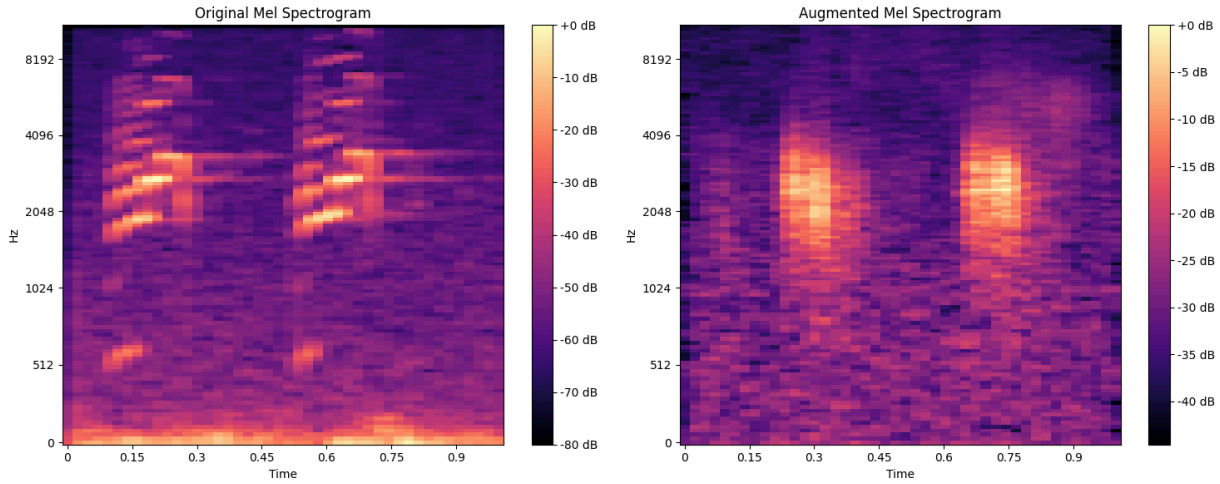

Figure 3.2 Mel Spectrogram Comparison of Original audio(left) vs generated audio(right) of species ‘amaevo’

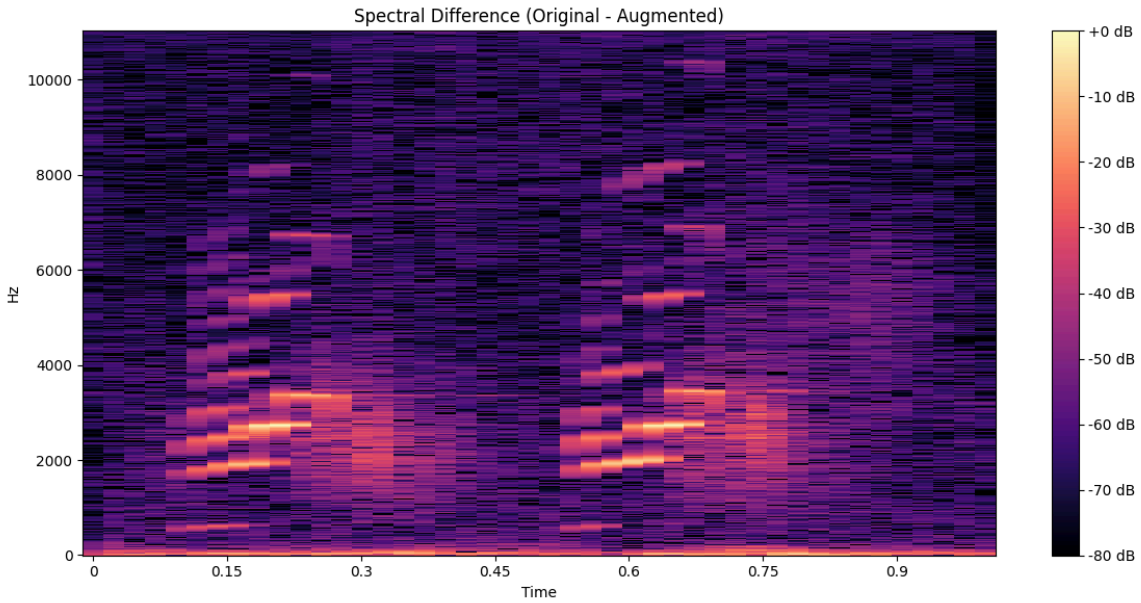

Figure 3.3 Spectral Difference of original vs generated audio species ‘Amaevo’

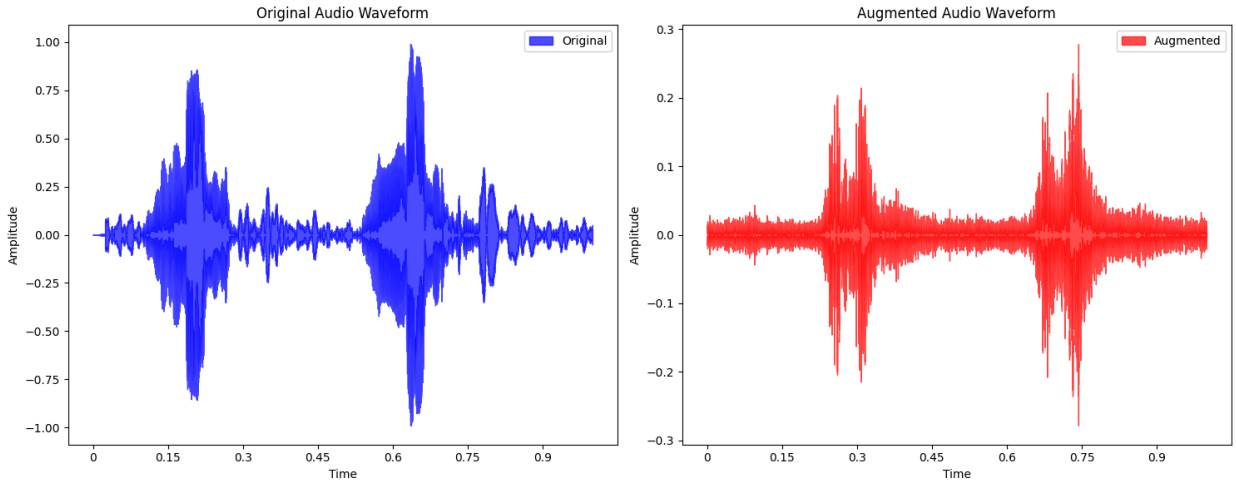

Figure 3.4 waveform comparison of original sound(left) vs generated sound(right)

3.8.2 Per-Species Average MSE (Noise Prediction)

Table 3.1

Species	MSE	Species	MSE
aldfly	0.007103	ameavo	0.014222
amebit	0.000474	amegfi	0.005362
amewig	0.001474	amtspa	0.012480
annhum	0.005961	astfly	0.001745
baleag	0.000317	balori	0.004363
banswa	0.004206	barswa	0.004211
belkin1	0.000796	belspa2	0.003065
bewwre	0.004356	bkbcuc	0.000087
bkbmag1	0.001130	bkbwar	0.000581
bkcchi	0.008810	bkchum	0.000159
bkhgro	0.002415	bktspa	0.000731
blkpho	0.003623	blugrb1	0.001700
bnhcow	0.006422	boboli	0.011528
bongul	0.008415	brebla	0.000069
brwhaw	0.003414	btbwar	0.082408

btnwar	0.023098	btywar	0.001098
buffle	0.000092	buhvir	0.006209
calqua	0.000631	camwar	0.000627
cangoo	0.006714	canwar	0.007538
canwre	0.006622	carwre	0.008332
caster1	0.002482	chispa	0.004645
chiswi	0.006429	chswar	0.001720
cliswa	0.003606	comgol	0.001179
comgra	0.004993	comloo	0.001415
commer	0.001191	comnig	0.008907
comyel	0.004875	coohaw	0.004344
coshum	0.005407	cowscj1	0.002813
daejun	0.004574	dusfly	0.000494
eargre	0.001500	easblu	0.001933
easkin	0.000869	easmea	0.005090
easpho	0.000118	eawpew	0.010829
eucdov	0.008551	eursta	0.005880
evegro	0.006921	fiespa	0.005472

fiscro	0.023627	foxspa	0.010685
gadwal	0.002876	gnttow	0.001111
gnwtea	0.004623	gocspa	0.000178
goleag	0.000058	grbher3	0.004293
greegr	0.005748	greroa	0.002287
grnher	0.000513	grtgra	0.003173
grycat	0.004337	gryfly	0.000350
hamfly	0.000656	hergul	0.001879
hoomer	0.000171	hoowar	0.005578
horlar	0.002098	houspa	0.004565
houwre	0.004298	indbun	0.003705
juntit1	0.000131	killde	0.010063
labwoo	0.001164	lazbun	0.003641
leabit	0.003514	leafly	0.001957
leasan	0.049609	lecthr	0.000990
lesnig	0.017508	lesyel	0.008658
linspa	0.001343	lobcur	0.003311
lobdow	0.000139	logshr	0.002001

lotduc	0.001105	mallar3	0.003643
merlin	0.005458	moublu	0.000179
mouchi	0.005484	moudov	0.003624
amecro	0.004563	amekes	0.007021
amepip	0.007741	amered	0.002790
amerob	0.006166	amewoo	0.011493
baisan	0.000179	bawwar	0.005654
bkpwar	0.005861	blujay	0.007567
brdowl	0.004792	brnthr	0.009840
brthum	0.000175	buggna	0.005533
bulori	0.004136	bushti	0.007982
buwwar	0.002636	cacwre	0.002128
calgul	0.009536	casfin	0.002792
casvir	0.018738	cedwax	0.000694
chukar	0.000232	clanut	0.005852
comrav	0.004465	comred	0.001582
comter	0.010665	dowwoo	0.003062
eastow	0.006029	gockin	0.009189

grcfly	0.008752	greyel	0.003891
grhowl	0.012447	haiwoo	0.004757
herthr	0.006537	houfin	0.013697
larspa	0.002352	lesgol	0.009631
lewwoo	0.000042	louwat	0.002437
macwar	0.004272	magwar	0.002434
marwre	0.007820	norcar	0.003697
norfli	0.003437	norhar2	0.000299
normoc	0.004239	norpar	0.002823
norpin	0.002598	norsho	0.004468
norwat	0.009991	nrwswa	0.002283
nutwoo	0.000625	olsfly	0.001259
orcwar	0.004047	osprey	0.006820
ovenbi1	0.002678	palwar	0.045262
pasfly	0.004797	pecsan	0.000203
perfal	0.005634	phaino	0.000248
pibgre	0.007659	pilwoo	0.005721
pingro	0.001266	pinjay	0.001036

pinsis	0.004244	pinwar	0.007177
plsvir	0.000955	prawar	0.005713
purfin	0.004548	pygnut	0.024716
rebmer	0.000145	rebnut	0.005377
rebsap	0.000712	rebwoo	0.003224
redcro	0.005017	reevir1	0.004426
renpha	0.000080	reshaw	0.002427
rethaw	0.001539	rewbla	0.004662
ribgul	0.001332	rinduc	0.000029
robgro	0.002486	rocpig	0.015683
rocwre	0.008789	rthhum	0.003112
ruckin	0.002825	rudduc	0.000090
rufgro	0.000045	rufhum	0.004007
rusbla	0.018257	sagspa1	0.000685
sagthr	0.000703	savspa	0.005390
saypho	0.000231	scatan	0.005720
scoori	0.001910	semplo	0.033601
semsan	0.007163	sheowl	0.000130

shshaw	0.001016	snobun	0.006490
snogoo	0.001912	solsan	0.000126
sonspa	0.007165	sora	0.003853
sposan	0.002991	spotow	0.003665
stejay	0.011239	swahaw	0.020133
swaspa	0.009287	swathr	0.006930
treswa	0.015677	truswa	0.000191
tuftit	0.008755	tunswa	0.007246
veery	0.012372	vesspa	0.007908
vigswa	0.030226	warvir	0.011265
wesblu	0.000692	wesgre	0.002396
weskin	0.000840	wesmea	0.005889
wessan	0.047113	westan	0.001126
wewpew	0.003573	whbnut	0.010805
whcspa	0.008400	whfibi	0.017390
whtspa	0.006755	whtswi	0.001275
wilfly	0.006319	wilsni1	0.002808
wiltur	0.008127	winwre3	0.010547

wlswar	0.005216	wooduc	0.005614
wooscj2	0.002256	woothr	0.007592
y00475	0.008432	yebfly	0.000415
yebsap	0.004030	yehbla	0.021313
yelwar	0.005568	yerwar	0.005541
yetvir	0.003937		

Table 3.8

REFERENCES

1. Kong et al., "HiFi-GAN: Generative Adversarial Networks for Efficient and High Fidelity Speech Synthesis," NeurIPS 2020.
2. Engel et al., "DDSP: Differentiable Digital Signal Processing," ICLR 2020.
3. Kong et al., "DiffWave: A Versatile Diffusion Model for Audio Synthesis," ICLR 2021.
4. Chen et al., "WaveGrad: Estimating Gradients for Waveform Generation," ICLR 2021.
5. Huang et al., "FastDiff: A Fast Conditional Diffusion Model for High-Quality Speech Synthesis," arXiv:2204.09934, 2022.
6. Dhariwal and Nichol, "Diffusion Models Beat GANs on Image Synthesis," NeurIPS 2021.
7. J. Engel, L. Hantrakul, C. Gu, and A. Roberts, "DDSP: Differentiable Digital Signal Processing," *International Conference on Learning Representations*, Apr. 2020, [Online]. Available: http://dblp.uni-trier.de/db/journals/corr/corr2001.html#abs-2001-04643
8. Liu, Yunyi, Craig Jin, and David Gunawan. "DDSP-SFX: Acoustically-guided sound effects generation with differentiable digital signal processing." arXiv preprint arXiv:2309.08060 (2023).
9. Wu, Zike, Pan Zhou, Kenji Kawaguchi, and Hanwang Zhang. "Fast diffusion model." arXiv preprint arXiv:2306.06991 (2023).
10. A. Guei, S. Christin, N. Lecomte, and É. Hervet, "ECOGEN: Bird sounds generation using deep learning," Methods in Ecology and Evolution, vol. 15, no. 1, pp. 69–79, Nov. 2023, doi: 10.1111/2041-210x.14239.
11. Dhariwal, Prafulla, et al. "Jukebox: A generative model for music." arXiv preprint arXiv:2005.00341 (2020). https://doi.org/10.48550/arXiv.2005.00341
12. Xeno-Canto Bird Recordings Extended (A-M). (2020). Additional recordings of 264 species from Xeno-Canto (part A-M). https://kaggle.com/rohanrao/xeno-canto-bird-recordings-extended-a-m

13. Xeno-Canto Bird Recordings Extended (N-Z). (2021). Additional recordings of 264 species from Xeno-Canto (part N-Z). https://kaggle.com/rohanrao/xeno-canto-bird-recordings-extended-n-z

14. Razavi, Ali, Aäron van den Oord, and Oriol Vinyals. "Generating Diverse High-Fidelity Images with VQ-VAE-2 (Supplementary Material)."

15. Ho, Jonathan, Ajay Jain, and Pieter Abbeel. "Denoising diffusion probabilistic models." Advances in neural information processing systems 33 (2020): 6840-6851.

16. Payne, C. (2019). MuseNet: A transformer-based model for generating music with multiple instruments and styles.

17. Razavi, A., van den Oord, A., & Vinyals, O. (2019). VQ-VAE-2: A hierarchical generative model for high-fidelity image synthesis. https://doi.org/10.48550/arXiv.1906.00446

18. Kaewtip, K., Gemmeke, J. F., & Van Hamme, H. (2013) Non-negative matrix factorization (NMF) for separating bird vocalizations from noisy audio.

19. Allen, J.B. & Rabiner, L.R. (1977). A unified approach to short-time Fourier analysis and synthesis. Proceedings of the IEEE.

20. Griffin, D. & Lim, J. (1984). Signal estimation from modified short-time Fourier transform. IEEE Transactions on Acoustics, Speech, and Signal Processing.

21. https://paperswithcode.com/method/vq-vae-2

22. https://keras.io/examples/generative/vq_vae/

23. [2307.12232] Signal Reconstruction from Mel-spectrogram Based on Bi-level Consistency of Full-band Magnitude and Phase

24. McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). Librosa: Audio and Music Signal Analysis in Python. *Proceedings of the 14th Python in Science Conference*. Retrieved from https://librosa.org